Lab 3: Query Tuning and Execution Plans

Preparation / Setup

0. Download AdventureWorks.bak.
1. Open SSMS and connect to your local SQL Server instance.
2. In Object Explorer, right-click Databases → Restore Database.
3. Select Device → Add the AdventureWorks.bak file from your file system → Click OK.
4. In SSMS: **Query → Include Actual Execution Plan** (or Ctrl + M).
5. Before running test queries run:

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
```

- SET STATISTICS IO ON reports logical and physical reads, helping measure I/O cost.
- SET STATISTICS TIME ON reports CPU time and elapsed execution time.

1. What is an Execution Plan?

An **execution plan** is a detailed roadmap showing how SQL Server processes a query including the order of operations, index usage, and estimated costs. It represents the **Query Optimizer's chosen strategy** for retrieving data.

Why Execution Plans Matter

- Provide a visual representation of how SQL Server retrieves data, including join order, index usage, and operator sequence.
- Identify performance bottlenecks such as table scans, key lookups, or expensive sort operations.
- Allow comparison of execution strategies before and after tuning or index changes.
- Display cost percentages per operator to reveal which parts of the query consume the most resources.

Estimated vs Actual plans:

- **Estimated Plan:** optimizer's prediction (no query execution)

- **Actual plan**: includes runtime metrics such as Actual row counts, Memory usage, Operator execution time.

2. Table Scan vs Clustered Index Scan

Concepts:

- **Table Scan:** reads every row (no useful index). Costly on large tables.

- A table without a clustered index is called a **heap** (Rows are stored in an unpredictable, unsorted physical order).

- **Clustered Index Scan:** reads table in clustered-index order. Still reads all rows but can be more efficient if physical order helps.

Create a test copy of the table without indexes

```
SELECT * INTO Sales.SalesOrderDetail_new
FROM Sales.SalesOrderDetail;

-- Table scan (no index)
SELECT * FROM Sales.SalesOrderDetail_new;

-- Clustered index scan (on indexed table)
SELECT * FROM Sales.SalesOrderDetail;
```

- Run both queries, capture STATISTICS IO/TIME and actual plans.

Observation: table copy will show a Table Scan and higher logical reads.

3. ORDER BY / GROUP BY Optimization Using Clustered Indexes

Sorting and grouping can be resource intensive if the database must perform an **explicit sort**. When the data is already ordered by an index, SQL Server can **skip the sort step** , reducing CPU and memory usage..

```
-- Heap table (no index)
SELECT * FROM Sales.SalesOrderDetail_new
ORDER BY SalesOrderID;

-- Indexed table (clustered index exists)
SELECT * FROM Sales.SalesOrderDetail
ORDER BY SalesOrderID;
```

- Sales.SalesOrderDetail_new table is a **heap**, so SQL Server must **sort the data** to apply ORDER BY.
- Sales.SalesOrderDetail has a **clustered index** on (SalesOrderID, SalesOrderDetailID), so rows are already **physically ordered** by the leading key (SalesOrderID), allowing the optimizer to return sorted results without an additional Sort operator.

If the column used in ORDER BY or GROUP BY aligns with the **leading column** of a clustered index, SQL Server can avoid sorting entirely, improving performance and lowering query cost.

4. Index Seek and Nonclustered Indexes

Concept:

An **Index Seek** is one of the fastest ways SQL Server retrieves data. It occurs when the query filter matches an **indexed column**, allowing SQL Server to jump directly to relevant rows instead of scanning the entire table.

A **Nonclustered Index** is a separate structure containing key columns and pointers to the base table.

By adding extra columns with the **INCLUDE** clause, you can create a **covering index**, eliminating costly **Key Lookup** operations.

A **Key Lookup** occurs when a nonclustered index finds matching rows but must fetch additional columns from the clustered index. This increases I/O and slows performance.

```
-- 1. Query without an index (results in a table scan)
SELECT ProductID, CarrierTrackingNumber, OrderQty
FROM Sales.SalesOrderDetail_new
WHERE CarrierTrackingNumber = '4911-403C-98';
```

Result: The execution plan shows a Table Scan because no index exists to support the filter condition.

Create Nonclustered Index: Create a nonclustered index on column CarrierTrackingNumber in the original table and include extra columns so that the query can be satisfied entirely from the index.

```
CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_CarrierTrackingNumber
ON Sales.SalesOrderDetail(CarrierTrackingNumber)
INCLUDE (ProductID, OrderQty);
```

Run the same query again on the original table:

```
SELECT ProductID, CarrierTrackingNumber, OrderQty
FROM Sales.SalesOrderDetail
WHERE CarrierTrackingNumber = '4911-403C-98';
```

Observation: - The execution plan now shows an **Index Seek**, directly navigates to the matching rows using the new index.

- Because the index includes ProductID and OrderQty, the query is fully covered (no Key Lookup).
- This significantly reduces **logical reads** and improves performance.

5. Avoid SELECT *

Using SELECT * retrieves **all columns**, even those not needed by the query. This increases **I/O cost, memory usage**, and may **prevent SQL Server from using an index**, leading to slower performance.

```
-- Inefficient
SELECT * FROM Sales.SalesOrderDetail
WHERE [CarrierTrackingNumber] = '4911-403C-98';

-- Efficient
SELECT CarrierTrackingNumber, ProductID
FROM Sales.SalesOrderDetail
WHERE CarrierTrackingNumber = '4911-403C-98';
```

The second query executes more efficiently, because it retrieves fewer columns and can use a covering index efficiently.

Best Practice: Always select only required columns to reduce data retrieval time and memory usage.

6. Limit Data Retrieval

When a query retrieves more rows than needed, it increases **I/O, CPU,** and **memory usage** unnecessarily. If your use case only requires a small subset, you can use the **TOP** keyword with **ORDER BY** to limit the result set efficiently.

If a proper index exists on the sorting column, SQL Server can stop reading once the required rows are found, avoiding a full table or index scan.

```
CREATE INDEX IX_SalesOrderDetail_LineTotal
ON Sales.SalesOrderDetail(LineTotal)
INCLUDE(ProductID); -- Create an index to support the ORDER BY clause

SELECT TOP 10 SalesOrderID, ProductID, LineTotal
FROM Sales.SalesOrderDetail
ORDER BY LineTotal;
```

The second query stops early after retrieving only 10 rows, resulting in lower I/O and faster execution time.

7. Aggregation Performance

```
SELECT ProductID,
       SUM(LineTotal)
FROM Sales.SalesOrderDetail
GROUP BY ProductID;
```

When executed without supporting indexes, SQL Server typically performs a **Clustered Index Scan** and a **Hash Aggregate**, resulting in higher I/O and CPU usage. This occurs because the underlying data is not sorted by ProductID, requiring additional work to group rows efficiently.

To improve performance, create a nonclustered index on the ProductID column (used in the GROUP BY clause) and include the LineTotal column (required by the aggregate function):

```
CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_ProductID
ON Sales.SalesOrderDetail(ProductID)
INCLUDE (LineTotal);
```

After indexing, verify whether the operator changes from **Hash Aggregate** to **Stream Aggregate** in the execution plan. Stream aggregates consume less memory and CPU when input data is sorted.

Aggregation Methods:

Aggregate Type	Description	When Used
Hash Aggregate	Builds a hash table in memory	When input data is unsorted
Stream Aggregate	Processes pre-sorted input efficiently	When input is indexed or ordered

8. Functions on Indexed Columns (SARGability)

Applying functions to indexed columns prevents SQL Server from using existing indexes, making the query **non-SARGable** (not Search ARGument Able).

```
SELECT *
FROM Production.Product
WHERE LEFT (Name, 1) = 'A'; -- results in table scan

-- SARGable: index can be leveraged for efficient filtering
SELECT ProductID, Name
FROM Production.Product
WHERE Name LIKE 'A%';
```

Avoid functions on indexed columns (`YEAR(), ISNULL(),Left(), Upper(),...`) in the WHERE clause, rewrite predicates to preserve index seeks.

9. Join Optimization

When executing joins, SQL Server selects the join order and algorithm based on table size, query predicates, and available indexes. Inadequate indexing can significantly increase I/O costs and lead to inefficient join operations.

```
SELECT soh.SalesOrderID, soh.OrderDate, sod.ProductID, sod.LineTotal
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesOrderDetail sod
    ON soh.SalesOrderID = sod.SalesOrderID
WHERE soh.OrderDate >= '2014-01-01'
  AND sod.LineTotal > 1000;
```

To tune this query, Create Supporting Indexes:

```
CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_OrderDate
ON Sales.SalesOrderHeader(OrderDate);

CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_LineTotal
```

```
ON Sales.SalesOrderDetail(LineTotal)
INCLUDE (ProductID);
```

Note: The column SalesOrderID is already part of the clustered primary key on both joined tables. Because clustered index key columns are automatically included in all nonclustered indexes, creating an additional index on SalesOrderID provides no performance benefit and is unnecessary.

Observe the Join Algorithm:

Before and after running the query and viewing the actual execution plan, check which join strategy SQL Server selected.

Join Type	Description	Best For
Nested Loops Join	Compares rows one by one	Small tables
Hash Match Join	Uses hash tables for matching	Large, unsorted tables
Merge Join	Merges sorted inputs	Large, pre-sorted tables

10. Composite Indexes for Multi-Condition Filters

When a query filters on multiple columns, a **composite (multi-column) index** can significantly improve performance by allowing SQL Server to seek efficiently instead of scanning.

```
SELECT ProductID, Name, ProductNumber, ListPrice, Weight
FROM Production.Product
WHERE ListPrice BETWEEN 500 AND 2000
AND Weight > 10;
```

To tune this query, create a **composite nonclustered** index on the filtered columns and include additional columns to avoid key lookup operations:

```
CREATE NONCLUSTERED INDEX IX_Product_ListPrice_Weight
ON Production.Product(ListPrice, Weight)
INCLUDE (ProductNumber, Name);
```

Step-by-Step Column Order Rules in Composite Indexes

1. Place columns with equality filters first (=) on the far left of the index key (eg. WHERE Col1 = 'Value').
    - In case of multiple equality filter columns, put the frequently used standalone filtering columns first.

2. For range or inequality filters (>) (BETWEEN, <, >, LIKE 'A%'), place columns in order of selectivity (the column that filters out more rows should appear first).

    - **Selectivity = Rows Matching Condition / Total Rows**
    - The **smaller the selectivity** value, **the more selective** the condition is.
    - Eg.
        i. `ListPrice BETWEEN 500 AND 2000 → Returns (101) rows → (0.2) Selectivity value → More Selective Column`
        ii. `Weight > 10 → Returns (126) rows → (0.25) Selectivity → Less Selective Column`

        - **More selective** column has a priority to be added **first** in the index key sequence.

3. ORDER BY and GROUP BY Columns

    - If your queries frequently sort or group by specific columns, place those columns next in the key sequence to avoid a separate sort operation in the execution plan.

4. Include additional columns only when necessary to cover the query.

```
-- Example of query with Equality & Non-Equality conditions
SELECT *
FROM Orders
WHERE CustomerID = 100
AND OrderDate >= '2026-01-01';

-- A suitable index can be..
CREATE NONCLUSTERED INDEX INDEX IX_Orders
ON Orders (CustomerID, OrderDate);
```

Why Column Order Matters Composite indexes are read **left to right**. SQL Server can only use the index efficiently if the first key column in the index contributes to filtering.

A highly selective first column allows the optimizer to:

- Eliminate non-qualifying rows early in the search process
- Perform an Index Seek rather than an Index Scan
- Reduce overall I/O, CPU consumption, and execution time

If the leading column has low selectivity, the optimizer may not benefit from the index and could choose a scan instead.

Example of How Column Order in Composite indexes **matters** in answering queries based on the Query condition:

Query condition	(Department, City)	(City, Department)
Department only	Good	Limited
City only	Limited	Good
Both columns	Good	Good

Index Cleanup

Execute the following cleanup script to remove all indexes created during this exercise and restore the database to its original state.

```
-- Drop indexes created during the lab (cleanup)
DROP INDEX IF EXISTS IX_SalesOrderDetail_CarrierTrackingNumber ON Sales.
SalesOrderDetail;
DROP INDEX IF EXISTS IX_SalesOrderDetail_LineTotal ON Sales.SalesOrderD
etail;
DROP INDEX IF EXISTS IX_SalesOrderDetail_ProductID ON Sales.SalesOrderD
etail;
DROP INDEX IF EXISTS IX_SalesOrderHeader_OrderDate ON Sales.SalesOrderH
eader;
DROP INDEX IF EXISTS IX_Product_ListPrice_Weight ON Production.Product;
```